Python

▼ ***What is Python ?***

→ Python is a ***high-level, interpreted programming language*** known for its simplicity and readability. It is widely used for web development, data science, artificial intelligence, machine learning, automation, scripting, and more. Python emphasizes code readability with its clean syntax and indentation-based structure.

Key Features of Python:

- ***Interpreted*** : Executes code line by line.
- ***Dynamically Typed*** : No need to declare variable types.
- ***Extensive Libraries*** : Has rich libraries like NumPy, Pandas, TensorFlow, Django, Flask, etc.

▼ ***Installation of Python ?***

STEPS :

1. Go to the official Python website: https://www.python.org/downloads/
2. Click on **"Download Python [latest version]"** (e.g., Python 3.x.x).
3. The installer (`.exe` file) will be downloaded.
4. Open **Command Prompt (cmd)**:
5. Type `cmd`, and press **Enter**.
6. Type the following command and press **Enter** : If Python is installed correctly, it will display the installed version.

" python --version "

▼ ***What is Python Indentation ?***

→ Indentation in Python refers to ***the spaces or tabs at the beginning of a line of code*** that define the structure of the program. Unlike other programming languages that use curly braces `{}` to mark code blocks, Python **strictly enforces indentation** to indicate blocks of code.

→ The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

▼ ***Python Variables ?***

→ A **variable** in Python is used to store data. It acts as a container that holds values, which can be changed during program execution.

▼ ***Declaring a Variable in Python :***

- Python **automatically assigns** the data type based on the value.
- No need to specify `int`, `float`, `string`, etc.

```
x = 10  # Integer
y = "Sanjai"  # String
z = 3.14  # Float
```

▼ ***Variable Names :***

Rules for Naming Variables

✅ **Valid Variable Names**:

1. Must start with a **letter (a-z, A-Z) or an underscore** `_`, but not a number.
2. Can contain **letters, numbers (0-9), and underscores** `_`.
3. Variable names are **case-sensitive** (`age` and `Age` are different).
4. Should not use **Python keywords** (e.g., `if`, `else`, `class`, `def`, etc.).

```
name = "Sanjai"  # Valid
_age = 22  # Valid
totalAmount = 500  # Valid
```

Best Practices for Naming Variables

- Use **descriptive** names (`student_name` instead of `s`).
- Follow **snake_case** (`first_name`, `total_amount`).
- Use **camelCase** if working with JavaScript-like syntax (`firstName`).

▼ ***Assign Multiple Values :***

→ Python allows assigning **multiple values** to multiple variables in one line.

Example : ***Assign Different Values***

```
a, b, c = 10, 20, 30
print(a, b, c)  # Output: 10 20 30
```

Example : ***Assign Same Value to Multiple Variables***

```
x = y = z = 100
print(x, y, z)  # Output: 100 100 100
```

Example : ***Unpacking a List or Tuple***

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x, y, z)  # Output: apple banana cherry
```

▼ ***Output Variables :***

→ You can display the value of variables using `print()`.

Example : ***Printing Variables***

```
name = "Sanjai"
age = 22
print(name)
print(age)
```

Example : ***Combining Strings and Variables***

```
name = "Sanjai"
print("Hello, " + name)  # Output: Hello, Sanjai
```

Example : ***Using `f-strings` (Recommended)***

```
age = 22
print(f"I am {age} years old.")  # Output: I am 22 years old.
```

Example : *Using* `format()`

```
name = "Sanjai"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
```

▼ **Global Variables :**

→ A **global variable** is a variable declared outside of a function and can be accessed anywhere in the program.

Example : *Global Variable*

```
x = "Python"  # Global variable

def my_function():
    print("I love " + x)

my_function()  # Output: I love Python
```

Using the `global` ***Keyword***

→ To modify a **global variable** inside a function, use the `global` keyword.

```
x = "Python"  # Global variable

def my_function():
    global x
    x = "JavaScript"  # Modify global variable

my_function()
print(x)  # Output: JavaScript
```

▼ ***Python Data Types ?***

→ A **data type** defines the type of value a variable holds. Python provides several built-in data types.

▼ ***Types of Data Types :***

Text Type	`str`
Numeric Types	`int` , `float` , `complex`
Sequence Types	`list` , `tuple` , `range`
Mapping Type	`dict`
Set Types	`set` , `frozenset`
Boolean Type	`bool`
Binary Types	`bytes` , `bytearray` , `memoryview`
None Type	`NoneType`

▼ ***Setting the Specific Data Type :***

Example	Data Type	OutPut
x = str("Sanjai Kannan G")	**str**	**Sanjai Kannan G - str**
x = int(22)	**int**	**22 - int**

x = float(22.5)	float	22.5 - float
x = complex(1j)	complex	1j - complex
x = list(("sanjai", "kannan", "gokul"))	list	["sanjai", "kannan", "gokul"] - list
x = tuple(("sanjai", "kannan", "gokul"))	tuple	("sanjai", "kannan", "gokul") - tuple
x = range(6)	range	range(0, 6) - range
x = dist("name" : "sanjai", "age" : 22)	dict	{'name': 'sanjai', 'age': 22} - dist
x = set(("apple", "banana", "cherry"))	set	{'apple', 'banana', 'cherry'} - set
x = frozenset(("apple", "banana", "cherry"))	frozenset	frozenset({'banana', 'apple', 'cherry'})
x = bool(5)	bool	True
x = bytes(5)	bytes	b'\x00\x00\x00\x00\x00'
x = bytearray(5)	bytearray	bytearray(b'\x00\x00\x00\x00\x00')
x = memoryview(bytes(5))	memoryview	<memory at 0x00D58FA0>

▼ ***Python Casting (Type Conversion) ?***

→ Python allows you to convert data from one type to another using **casting functions**.

1. Casting to Integer (`int`) :

→ Converts values to an integer (removes decimals, converts strings if valid).

```
x = int(3.9)      # 3
y = int("10")     # 10
z = int(True)     # 1
```

2. Casting to Float (`float`) :

→ Converts values to a floating-point number.

```
x = float(5)      # 5.0
y = float("3.14")  # 3.14
z = float(False)   # 0.0
```

3. Casting to String (`str`) :

→ Converts values to a string.

```
x = str(100)     # "100"
y = str(3.14)    # "3.14"
z = str(True)    # "True"
```

4. Casting to Boolean (`bool`) :

→ Converts values to `True` or `False`.

- `0`, `None`, `""`, `[]`, `{}` → **False**
- Everything else → **True**

```
x = bool(0)       # False
y = bool(10)      # True
z = bool("Hello")  # True
```

5. Casting to List (`list`) :

→ Converts an iterable (tuple, string, range) to a list.

```
x = list("hello")      # ['h', 'e', 'l', 'l', 'o']
y = list((1, 2, 3))    # [1, 2, 3]
z = list(range(3))     # [0, 1, 2]
```

6. Casting to Tuple (`tuple`) :

→ Converts an iterable to a tuple.

```
x = tuple([1, 2, 3])   # (1, 2, 3)
y = tuple("hello")     # ('h', 'e', 'l', 'l', 'o')
```

7. Casting to Set (`set`) :

→ Converts an iterable to a set (removes duplicates).

```
python
CopyEdit
x = set([1, 2, 2, 3])   # {1, 2, 3}
y = set("banana")       # {'a', 'b', 'n'}
```

8. Casting to Dictionary (`dict`) :

→ Converts a list of key-value pairs to a dictionary.

```
x = dict([("name", "Sanjai"), ("age", 25)])
# {'name': 'Sanjai', 'age': 25}
```

▼ ***Python Strings ?***

▼ ***Strings :***

→ A **string** in Python is a sequence of characters enclosed in **single**, **double**, or **triple quotes**.

```
print("Sanjai") #Double Quotes
print('Sanjai')  #Single Quotes
```

▼ ***Slicing Strings :***

→ Python allows **indexing** and **slicing** strings using `[]`.

```
text = "Sanjai Kannan G"

# Accessing characters
print(text[0])    # S (first character)
print(text[-1])   # G (last character)

# Slicing [start:end:step]
print(text[0:6])   # Python (0 to 5)
print(text[7:])    # Kannan G (from index 7 to end)
print(text[:6])    # Sanja (start to index 5)
print(text[::2])   # Sna annG (every second character)
print(text[::-1])  # G nannaK iajnaS (Reverse string)
```

▼ ***Modify Strings :***

→ Python provides multiple string modification methods.

```
text = " Sanjai Kannan G"

# Changing case
print(text.upper())    # " HELLO WORLD "
print(text.lower())    # " hello world "
print(text.title())    # " Hello World "
print(text.capitalize()) # " Hello world "

# Removing spaces
print(text.strip())   # "hello world"
print(text.lstrip())  # "hello world " (removes left spaces)
print(text.rstrip())  # " hello world" (removes right spaces)

# Replacing
print(text.replace("hello", "Hi"))  # " Hi world "
```

▼ ***Concatenate Strings***

→ In Python, you can concatenate strings using the `+` operator or using the `join()` method.

```
# USING " + "

str1 = "sanjai"
str2 = "kannan"

result = str1 + " " + str2   # Concatenate with a space
print(result)  # Output: sanjai kannan
```

```
# USING " JOIN "

str1 = "sanjai"
str2 = "kannan"

result = " ".join([str1, str2])  # Join with a space
print(result)  # Output: sanjai kannan
```

▼ ***Format Strings***

→ In 2 Methods we can format a string in python.

1. **Using f-strings** (Python 3.6+)
2. **Using** `.format()`

```
# Using f-strings

str1 = "sanjai"
```

```
result = f"my name is {str1}"
print(result)
```

```
# Using .format()

name = "sanjai"
age = 22

result = "my name is {} and my age is {}".format(name, age)
print(result)
```

▼ ***Escape Characters***

→ Escape characters allow you to insert special characters into strings. For example:

- `\n`: Newline
- `\t`: Tab
- `\\`: Backslash
- `\'`: Single quote
- `\"`: Double quote

```
str1 = "Sanjai\nKannan"  # Newline between Sanjai and Kannan
print(str1)  # Output:
# Sanjai
# Kannan

str2 = "This is a \"quoted\" word."
print(str2)  # Output: This is a "quoted" word.

str3 = "Backslash: \\"
print(str3)  # Output: Backslash: \
```

▼ ***String Methods***

PYTHON STRING METHODS

No	Method	Description	Example
1	capitalize()	Converts the first character to upper case	`"hello".capitalize()` → `'Hello'`
2	casefold()	Converts string into lower case	`"HELLO".casefold()` → `'hello'`
3	center()	Returns a centered string	`"hello".center(10, "*")` → `'**hello***'`
4	count()	Returns the number of times a specified value occurs	`"hello".count('l')` → `2`
5	encode()	Returns an encoded version of the string	`"hello".encode()` → `b'hello'`
6	endswith()	Returns true if the string ends with the specified value	`"hello".endswith('o')` → `True`

No	Method	Description	Example
7	expandtabs()	Sets the tab size of the string	`"hello\tworld".expandtabs(4)` → `'hello world'`
8	find()	Searches the string for a specified value	`"hello".find('e')` → `1`
9	format()	Formats specified values in a string	`"My name is {}".format("Alice")` → `'My name is Alice'`
10	format_map()	Formats specified values in a string	`"My name is {name}".format_map({"name": "Alice"})` → `'My name is Alice'`
11	index()	Searches the string for a specified value	`"hello".index('e')` → `1`
12	isalnum()	Returns True if all characters are alphanumeric	`"hello123".isalnum()` → `True`
13	isalpha()	Returns True if all characters are in the alphabet	`"hello".isalpha()` → `True`
14	isascii()	Returns True if all characters are ascii characters	`"hello".isascii()` → `True`
15	isdecimal()	Returns True if all characters are decimals	`"123".isdecimal()` → `True`
16	isdigit()	Returns True if all characters are digits	`"123".isdigit()` → `True`
17	isidentifier()	Returns True if the string is an identifier	`"my_var".isidentifier()` → `True`
18	islower()	Returns True if all characters are lower case	`"hello".islower()` → `True`
29	isnumeric()	Returns True if all characters are numeric	`"123".isnumeric()` → `True`
20	isprintable()	Returns True if all characters in the string are printable	`"hello".isprintable()` → `True`
21	isspace()	Returns True if all characters are whitespaces	`" ".isspace()` → `True`
22	istitle()	Returns True if the string follows the rules of a title	`"Hello World".istitle()` → `True`
23	isupper()	Returns True if all characters are upper case	`"HELLO".isupper()` → `True`
24	join()	Joins the elements of an iterable to the end of the string	`"-".join(['hello', 'world'])` → `'hello-world'`
25	ljust()	Returns a left justified version of the string	`"hello".ljust(10, '*')` → `'hello*****'`
26	lower()	Converts a string into lower case	`"HELLO".lower()` → `'hello'`
27	lstrip()	Returns a left trim version of the string	`" hello".lstrip()` → `'hello'`
28	maketrans()	Returns a translation table to be used in translations	`"abc".maketrans("a", "z")` → `translation table`
29	partition()	Returns a tuple where the string is parted into three parts	`"hello world".partition(" ")` → `('hello', ' ', 'world')`
30	replace()	Returns a string where a specified value is replaced	`"hello".replace("e", "a")` → `'hallo'`
31	rfind()	Searches the string for a specified value (last occurrence)	`"hello".rfind('l')` → `3`
32	rindex()	Searches the string for a specified value (last occurrence)	`"hello".rindex('l')` → `3`
33	rjust()	Returns a right justified version of the string	`"hello".rjust(10, '*')` → `'*****hello'`
34	rpartition()	Returns a tuple where the string is parted into three parts (from right)	`"hello world".rpartition(" ")` → `('hello', ' ', 'world')`

No	Method	Description	Example
35	rsplit()	Splits the string at the specified separator, returns a list	`"a,b,c".rsplit(',')` → `['a', 'b', 'c']`
36	rstrip()	Returns a right trim version of the string	`"hello ".rstrip()` → `'hello'`
37	split()	Splits the string at the specified separator, returns a list	`"a,b,c".split(',')` → `['a', 'b', 'c']`
38	splitlines()	Splits the string at line breaks and returns a list	`"hello\nworld".splitlines()` → `['hello', 'world']`
39	startswith()	Returns true if the string starts with the specified value	`"hello".startswith('h')` → `True`
40	strip()	Returns a trimmed version of the string	`" hello ".strip()` → `'hello'`
41	swapcase()	Swaps cases, lower case becomes upper case and vice versa	`"Hello".swapcase()` → `'hELLO'`
42	title()	Converts the first character of each word to upper case	`"hello world".title()` → `'Hello World'`
43	translate()	Returns a translated string	`"abc".translate({97: 65})` → `'Abc'`
44	upper()	Converts a string into upper case	`"hello".upper()` → `'HELLO'`
45	zfill()	Fills the string with a specified number of 0 values at the beginning	`"42".zfill(5)` → `'00042'`

▼ ***Python Booleans ?***

Boolean Values :

→ In programming you often need to know if an expression is `True` or `False`.

→ You can evaluate any expression in Python, and get one of two answers, `True` or `False`.

→ When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
print(10 > 9)
print(10 == 9)
print(10 < 9)

# Output
True
False
```

Evaluate Values and Variables :

→ The `bool()` function allows you to evaluate any value, and give you `True` or `False` in return,

```
print(bool("Hello"))
print(bool(15))

True
True
```

Most Values are True :

→ Almost any value is evaluated to `True` if it has some sort of content.

→ Any string is `True`, except empty strings.

→ Any number is `True`, except `0`.

→ Any list, tuple, set, and dictionary are `True`, except empty ones.

```
print(bool("abc"))
print(bool(123))
print(bool(["apple", "cherry", "banana"]))

# Output
True
True
True
```

Some Values are False :

→ In fact, there are not many values that evaluate to `False`, except empty values, such as `()`, `[]`, `{}`, `""`, the number `0`, and the value `None`. And of course the value `False` evaluates to `False`.

```
print(bool(False))
print(bool(None))
print(bool(0))
print(bool(""))
print(bool(()))
print(bool([]))
print(bool({}))

# Output
False
False
False
False
False
False
False
```

▼ ***Python Operators ?***

→ Operators are used to perform operations on variables and values.

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Python Arithmetic Operators

→ Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Python Assignment Operators

→ Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
`	=`	`x
^=	x ^= 3	x = x ^ 3
>≥	x >>= 3	x = x >> 3
<≤	x <<= 3	x = x << 3
:=	x := 3	x = 3

Python Comparison Operators

→ Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
≠	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
≥	Greater than or equal to	x >= y
≤	Less than or equal to	x <= y

Python Logical Operators

→ Logical operators are used to combine conditional statements:

Operator	**Description**	**Example**
and	Returns “True” if both statements are true	x < 5 and x < 10
or	Returns “True” if at least one of the statements is true	x < 5 or x < 4
not	Reverses the result, returns “False” if the result is true	not(x < 5 and x < 10)

Python Identity Operators

→ Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns “True” if both variables point to the same object	x is y
is not	Returns “True” if both variables do not point to the same object	x is not y

Python Membership Operators

→ Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns “True” if a sequence with the specified value is present in the object	x in y
not in	Returns “True”if a sequence with the specified value is not present in the object	x not in y

Python Bitwise Operators

→ Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to `1` if both bits are `1`	`x & y`
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

▼ *Python Lists ?*

List :

→ Lists are used to store multiple items in a single variable. Lists are created using square brackets:

```
# Syntax
names = ["sanjai", "kannan", "gokul"]
print(names)

# Output
['sanjai', 'kannan', 'gokul']
```

List Items :

→ List items are ordered, changeable, and allow duplicate values. List items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered :

→ When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

→ If you add new items to a list, the new items will be placed at the end of the list.

Changeable :

→ The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates :

→ Since lists are indexed, lists can have items with the same value.

▼ ***Access List Items :***

Access Items :

→ List items are indexed and you can access them by referring to the index number:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[1])

# Output
kannan
```

Negative Indexing :

→ Negative indexing means start from the end

→ `1` refers to the last item, `2` refers to the second last item etc.

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[-1])

# Output
suja
```

Range of Indexes :

→ You can specify a range of indexes by specifying where to start and where to end the range.

→ When specifying a range, the return value will be a new list with the specified items.

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[1:4])

# Output
['kannan', 'gokul', 'Kannan']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[:2])

# Output
['sanjai', 'kannan']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[2:])
```

```
# Output
['gokul', 'Kannan', 'guna', 'suja']
```

Range of Negative Indexes :

→ Specify negative indexes if you want to start the search from the end of the list:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[-4:-1])

# Output
['gokul', 'Kannan', 'guna']
```

Check if Item Exists :

→ To determine if a specified item is present in a list use the `in` keyword:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
if "gokul" in names:
   print("yes")
else:
   print("no")

# Output
yes
```

▼ ***Change List Items :***

Change Value :

→ To change the value of a specific item, refer to the index number:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1] = "kannanChanged"
print(names)

# Output
['sanjai', 'kannanChanged', 'gokul', 'Kannan', 'guna', 'suja']
```

Change a Range of Item Values :

→ To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1:3] = ["kannanChanged", "gokulChanged"]
print(names)
```

```
# Output
['sanjai', 'kannanChanged', 'gokulChanged', 'Kannan', 'guna', 'suja']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1] = ["kannanChanged", "kannanChanged"]
print(names)

# Output
['sanjai', ['kannanChanged', 'kannanChanged'], 'gokul', 'Kannan', 'guna', 'suja']
```

Insert Items :

→ To insert a new list item, without replacing any of the existing values, we can use the `insert()` method.

→ The `insert()` method inserts an item at the specified index:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.insert(2, "kannanNameChangedToSK")
print(names)

# Output
['sanjai', 'kannan', 'kannanNameChangedToSK', 'gokul', 'Kannan', 'guna', 'suja']
```

▼ ***Add List Items :***

Append Items :

→ To add an item to the end of the list, use the append() method:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.append("appendedName")
print(names)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'appendedName']
```

Insert Items :

→ To insert a list item at a specified index, use the `insert()` method.

→ The `insert()` method inserts an item at the specified index:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.insert(1, "InsertedName")
print(names)

# Output
['sanjai', 'InsertedName', 'kannan', 'gokul', 'Kannan', 'guna', 'suja']
```

Extend List :

→ o append elements from *another list* to the current list, use the `extend()` method.

```
first_name_list = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
second_name_list = ["Ravi", "John", "Doe"]
first_name_list.extend(second_name_list)
print(first_name_list)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'Ravi', 'John', 'Doe']
```

Add Any Iterable :

→ The `extend()` method does not have to append *lists*, you can add any iterable object (tuples, sets, dictionaries etc.).

```
first_name_list = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
second_name_list = ("Ravi", "John", "Doe")
first_name_list.extend(second_name_list)
print(first_name_list)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'Ravi', 'John', 'Doe']
```

▼ ***Remove List Items :***

Remove Specified Item:

→ The `remove()` method removes the specified item.

```
names = [ "sanjai", "kannan", "gokul" ]
names.remove("sanjai")
print(names)

# Output
['kannan', 'gokul']
```

Remove Specified Index :

→ The `pop()` method removes the specified index.

```
names = [ "sanjai", "kannan", "gokul" ]
names.pop()         # Remove the last item
print(names)

# Output
['sanjai', 'kannan']
```

```
names = [ "sanjai", "kannan", "gokul" ]
names.pop(1)          # Remove the second item
print(names)

# Output
['sanjai', 'gokul']
```

→ The `del` keyword also removes the specified index:

```
names = [ "sanjai", "kannan", "gokul" ]
del names[0]
print(names)

# Output
['kannan', 'gokul']
```

→ The `clear()` method empties the list. The list still remains, but it has no content.

```
names = [ "sanjai", "kannan", "gokul" ]
names.clear()
print(names)

# Output
[]
```

▼ ***Loop List :***

Loop Through a List :

→ You can loop through the list items by using a `for` loop:

```
names = [ "sanjai", "kannan", "gokul" ]
for x in names:
   print(x)

# Output
sanjai
kannan
gokul
```

Loop Through the Index Numbers :

→ You can also loop through the list items by referring to their index number.

→ Use the `range()` and `len()` functions to create a suitable iterable

```
names = [ "sanjai", "kannan", "gokul" ]
for x in range(len(names)):
```

```
    print(x)

# Output
0
1
2
```

Using a While Loop :

→ You can loop through the list items by using a `while` loop.

→ Use the `len()` function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.

→ Remember to increase the index by 1 after each iteration.

```
names = [ "sanjai", "kannan", "gokul" ]
i = 0
while i < len(names):
  print(names[i])
  i = i + 1

# Output
sanjai
kannan
gokul
```